Análise Longitudinal de Hotspots em Softwares Públicos Brasileiros (2020-2026)

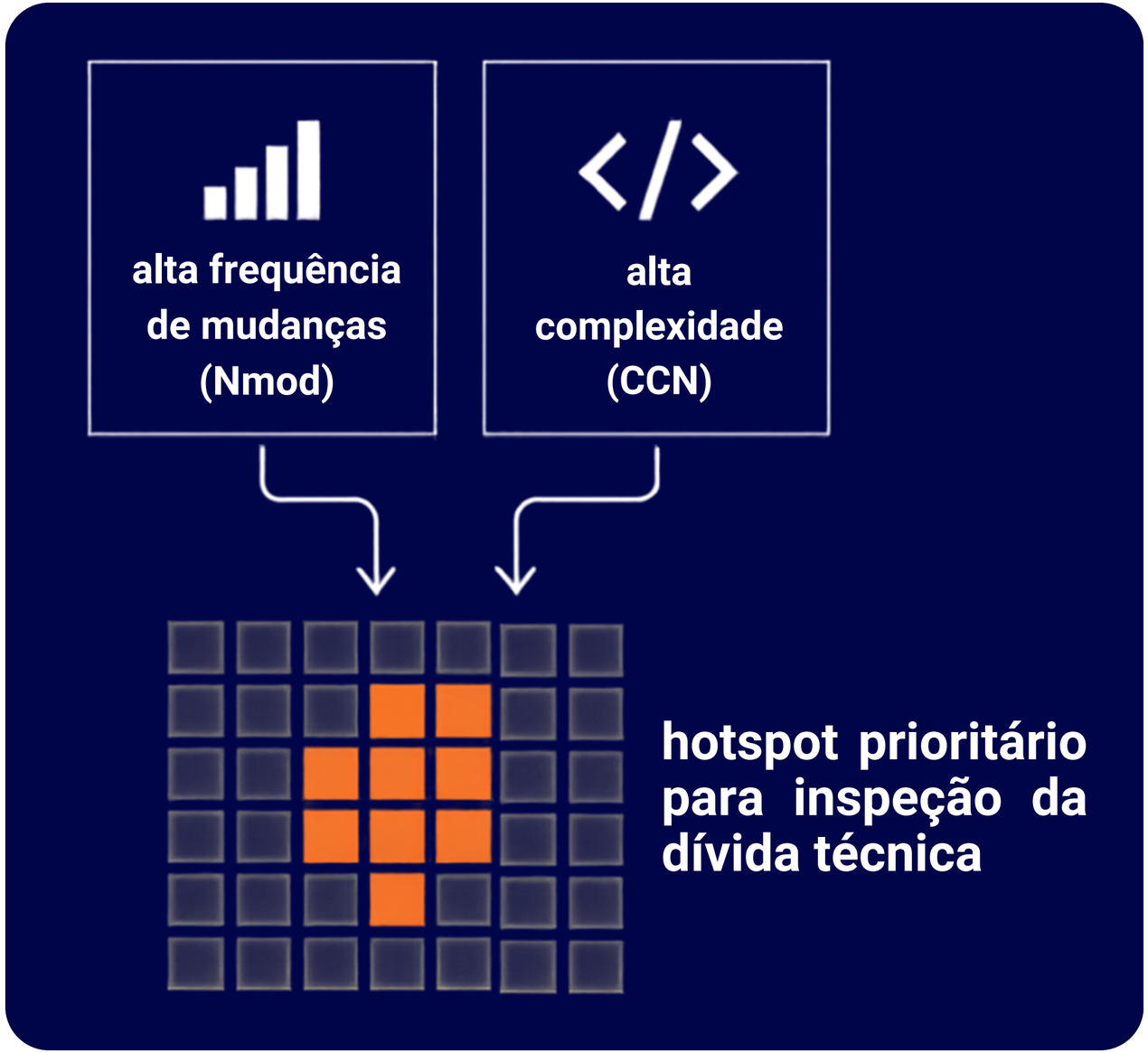
Hotspots de manutenção e complexidade em arquivos de código-fonte no histórico longitudinal de cinco softwares públicos brasileiros.

Alunos: Alan Mathias, Domini Acco, Guilherme Tombini, Jemison Santos.

Professores: Camila, Renato.

PROBLEMA DE PESQUISA

Investigar se arquivos com alta frequência de mudanças e alta complexidade, concentrados e persistentes, podem indicar regiões onde priorizar a inspeção do projeto.



OBJETIVOS

GERAL:

- **Avaliar hotspots como mecanismo de priorização da inspeção de dívida técnica.**

ESPECÍFICOS:

- **Medir manutenção e complexidade em arquivos;**
- **Analisar associação entre métricas Nmod, churn, NLOC e CCN;**
- **Identificar persistência temporal dos hotspots;**
- **Inspecionar qualitativamente até 15 hotspots selecionados;**

MÉTODO

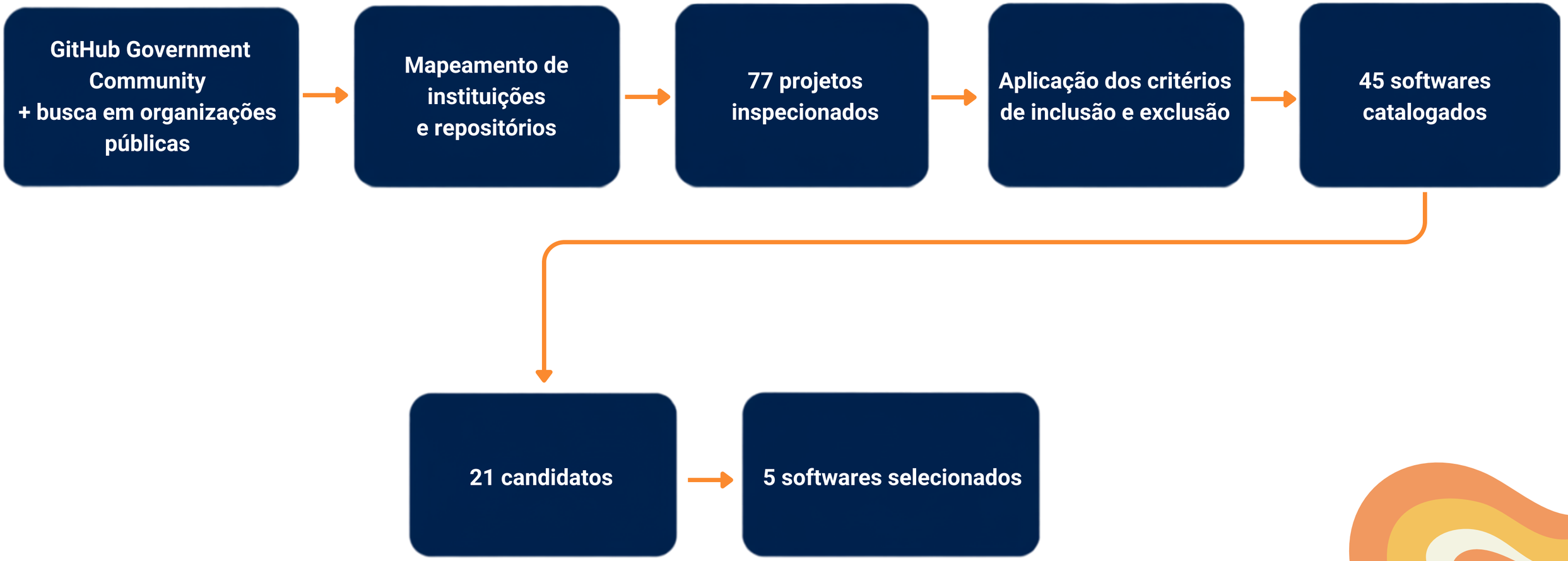

Git
extração com PyDriller
análise de complexidade com Lizard
cálculo das métricas por arquivo
Nmod, churn, NLOC e CCN
análise de concentração, associação e persistência
hotspots persistentes
inspeção qualitativa (até 15 casos)

COLETA DE REPOSITÓRIOS



- **Unidade de análise:** software, não repositório.
- **Exclusões:** APIs, datasets, ETLs, bibliotecas, documentação e projetos sem histórico suficiente.

FLUXO DO PROCESSO



SOFTWARES SELECIONADOS



A seleção buscou diversidade de instituição, domínio, porte, maturidade e tecnologia.

MÉTRICAS MÍNIMAS



Nmod

número de commits que modificaram o arquivo.



Churn

linhas adicionadas + removidas.



NLOC

tamanho do arquivo.



CCN

complexidade ciclomática.



Regra operacional do hotspot: top 20% de Nmod e (interseção) do top 20% de CCN.

REFERÊNCIAS:

- Sutoyo et al. (2025) — evolução de dívida arquitetural, mudanças por arquivo, PyDriller e Lizard.
- Li et al. (2015) — taxonomia e gestão de dívida técnica.
- Alves et al. (2016) — identificação e indicadores de dívida técnica.
- Nagappan & Ball (2005) — code churn e esforço/risco de manutenção.
- McCabe (1976) — definição da complexidade ciclomática.
- Potdar & Shihab (2014) — Self-Admitted Technical Debt e evidências em artefatos textuais.
- Venters et al. (2023) — sustentabilidade técnica, manutenção e evolução de software.